ADDING ORDER HISTORY TO YOUR EXPERIENCE

Last Updated: 10/5/2018

Submit Feedback: Dev Portal Slack channel #devportal

In This Guide:

Overview

Use this guide to add order history to your experience. Step-by-step instructions and code samples give you the tools you need to retrieve order information for your customers through your app.

TIP: Before using this guide you should have already completed Adding Checkout to Your

Experience.

What is an Order?

A Nike order consists of the following:

- Consumer's purchased items/services
- Consumer's payment method(s) and billing address(es)
- Consumer's shipping method(s) and shipping address(es)
- Pricing
- Taxes
- Discounts

[Insert diagram of high-level steps in the process]

1. Get a list of a customer's orders
2. Get the details of a customer's order

Step 1: Get a list of a customer's orders

[Overview Description] Describe Edge router

Get the customer's upmid if logged in; guest's email address. Get the unite authorization token in the form of 'bearer token' Send both upmid and authorization as headers

Understanding Order Status

Customizing Your Results

Filtering

Sorting*

You can sort the customer's orders in several ways using the `sort` query parameter.

Step 2: Get the details of a customer's order

[Overview Description]

required headers (guest) x-nike-visitorid x-nike-visitid appId

required path parameter order id

https://api.nike.com/order_mgmt/user_order_details/v1/C00000554850?filter=email(jane.moore@nike.com) if email is missing, get 404

API Endpoint Quick Reference

Order Summary Get a list of a customer's orders

Order Details Get the details of a customer's order

Sending Your First Request

For your first request, send a request to the *Create or Update a Cart by Cart ID* endpoint of the Carts v2 API and create your first cart.

1. Gather Info for the Request

Our example endpoint, *Create or Update a Cart by Cart ID*, only supports the HTTP PUT method. To create a cart, send a PUT request with (at minimum) the required request headers and the required parts of the request body.

First, read the Using Carts section of this document to learn more about the required parts of this request.

Assume that user for whom you are creating the cart is a Nike+ member who has logged in with their Nike+ credentials. This indicates which request headers are required.

For the request headers, the following considerations apply (at minimum):

1. Always send `application/json` in both the **Accept** and **Content-Type** headers.

2. Send the user's access token as obtained from Unite services in the **Authorization** header.

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer {your access token}
```

For the request body, the following considerations apply (at miniumum):

1. Send a UUID **that you have created** in the **id** field, in this case `61bc115b-16e5-43b5-bcaf-dd6168c543f8`. This is the cart ID.
2. Send the country code of the country where the user is shopping in the **country** field, in this case, `US`. Send `NIKE` in the **brand** field.
3. Send a UUID **that you have created, different from above** in the items.**id** field. This is the identifier for the line item in the cart. If you include multiple line items, each must have a unique identifier.
4. Send a valid SKU identifier obtained from the Merchandised Products SKU service in the items.**skuId** field.

```
{
  "id": "61bc115b-16e5-43b5-bcaf-dd6168c543f8",
  "country": "US",
  "brand": "NIKE",
  "items": [
    {
      "id": "9992b8cb-e4ac-42af-
a8bb-3454d8509d32",
      "skuId":
"1f6b36bd-61c0-5eb5-b9f0-8643e1ebae37",
      "quantity": 1
    }
```

```
  ]
}
```

2. Create the URL

The API.md states that the required URL format is `/buy/carts/v2/{id}`.

To build the full URL, prepend `https://api.nike.com` to the above path, then append **id** after “v2”. The **id** is the cart identifier you passed in the request body.

The complete URL is then https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8.

3. Execute the request

Execute the request with a cURL command. Using the values gathered in steps 1 and 2, the final cURL command is:

```
curl -X PUT \
  https://api.nike.com/buy/carts/v2/
61bc115b-16e5-43b5-bcaf-dd6168c543f8 \
  -H 'Accept: application/json' \
  -H 'Authorization: Bearer {your access
token}' \
  -H 'Cache-Control: no-cache' \
  -H 'Content-Type: application/json' \
  -d '{
     "id": "61bc115b-16e5-43b5-bcaf-
dd6168c543f8",
     "country": "US",
     "brand": "NIKE",
     "items": [
       {
```

```
        "id": "9992b8cb-e4ac-42af-
a8bb-3454d8509d32",
        "skuId":
"1f6b36bd-61c0-5eb5-b9f0-8643e1ebae37",
        "quantity": 1
    }
]
}
```

4. Parse the Response

Assuming no errors, you will receive a response body similar to the following:

```
{
    "id": "61bc115b-16e5-43b5-bcaf-
dd6168c543f8",
    "country": "US",
    "currency": "USD",
    "brand": "NIKE",
    "totals": {
        "subtotal": 130,
        "discountTotal": 0,
        "valueAddedServicesTotal": 0,
        "total": 130,
        "quantity": 1
    },
    "items": [
        {
            "id": "9992b8cb-e4ac-42af-
a8bb-3454d8509d32",
            "skuId":
"1f6b36bd-61c0-5eb5-b9f0-8643e1ebae37",
            "quantity": 1,
            "priceInfo": {
```

Some values in the response are exactly what you sent in the request, but the values in the **totals** section provide a cart pricing summary and the values in items.**priceInfo** section provide the current item pricing details.

Another Example

For your next request, send a GET request to the *Get a Cart for a Cart ID* endpoint of the Carts v2 API, using the cart **id** that you created in the previous step.

Refer to the Using Carts v2 section of this document to find the required parts of this request.

For the request headers, use the same headers you used in the previous step.

> Note: There is no request body needed for a GET request.

The complete URL is https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8. Note that the same cart **id** that you created for the previous PUT request is at the end of the URL.

The final cURL is:

```
curl -X GET \
  https://api.nike.com/buy/carts/v2/
61bc115b-16e5-43b5-bcaf-dd6168c543f8 \
  -H 'accept: application/json' \
  -H 'authorization: Bearer {your access
token}' \
  -H 'cache-control: no-cache' \
  -H 'content-type: application/json'
```

The response body from the *Get a Cart for a Cart ID* endpoint is the same as *Create or Update a Cart by Cart ID*, so the process of parsing it is also the same.

Best Practices

Conditions for Retries

For retry information by Checkout endpoint, visit Retry Patterns for Checkout Clients in Confluence.
For all Checkout APIs, the general rule is that HTTP 4XX error codes (except for 429) should not be retried but HTTP 5XX errors can be retried. For general information on Nike error retry practices, see API Error Patterns on Confluence.

Honor the ETAs for Best Performance

For async endpoints that return an ETA, i.e. the estimated time for the job to be completed, it is important for your app to honor the ETA for best performance. For example, if the ETA is 2000ms, your app should wait 2000ms to start polling the /jobs endpoint to avoid consuming network and other resources unnecessarily.

Test Scenarios

Here is an example list of test scenarios for a user experience that is integrating with the Checkout APIs. Note: in this context, inline refers to a regular Nike product as opposed to a customized Nike iD product.

- Single Item (Inline) - Single Payment (Credit Card)
- Single Item (Inline) - Single Payment (Gift Card)
- Single Item (Inline) - Single Payment (PayPal)
- Single Item (NikeID) - Single Payment (Credit Card)
- Single Item (NikeID) - Single Payment (PayPal)
- Single Item (Inline) - Multiple Payment (Credit Card & Gift Card)
- Single Item (NikeID) - Multiple Payment (Credit Card & Gift Card)
- Mixed Items (Inline & NikeID) - Single Payment (Credit Card)

- Mixed Items (Inline & NikeID) - Single Payment (PayPal)
- Mixed Items (Inline & NikeID) - Multiple Payment (Credit Card & Gift Card)

Additionally, it's useful to add scenarios for multi-quantity (i.e. quantity > 1) for Inline items, as well as scenarios with multiple Nike iD items in same checkout.

TIP: While inspecting browser activity on www.nike.com/launch, you can change your shopping country with the flag icon at the upper right of the homepage. Also, as necessary you can place an order to observe all the checkout calls. Orders can be cancelled via self-service within 30 minutes of submission, otherwise contact Nike Customer Service.

Test Environment

It is recommended to test all Checkout endpoints in the production environment as opposed to the test environment. Using the test environment can have unpredictable results due to the many downstream services which these endpoints are reliant upon in order to provide typical 'production-like' responses.

There are boundaries for testing in production:

- Performance tests at high volumes should never be done in production.
- Calling 'Request a Checkout Submit' in production can result in actual Nike orders being sent for fulfillment, and actual payment methods being authorized and/or charged. Proceed with caution.

Caching Data

None of the endpoints described in this document support caching.

Error Handling: Which JSON Field Had The Error?

In error responses from APIs, Nike uses the JSON Pointer standard to indicate which field of the request JSON had the error.

However, not all Checkout APIs are the same in this regard. This is due to some APIs having been built before Nike decided to use JSON Pointer standard.

For example, error responses from the Checkouts v2 API will vary from error responses from the Carts API.

Examples of both:

Checkouts (built before adopting JSON Pointer standard):

```
{
    "message": "Validation Failed",
    "errors": [{
        "field":
"request.items[0].contactInfo.email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

Carts (built after adopting JSON Pointer standard):

```
{
    "message": "Validation Failed",
```

```
    "errors": [{
        "field": "/request/items/0/contactInfo/
email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

If you are a client of both of the above APIs, you will need to parse error responses in two different ways.

Troubleshooting

TIP: SLAs vary per endpoint for many of the Buy APIs. In the figures listed above, the highest response time and lowest requests per second *for the API overall* were shown. Ask the Product Owner to get specific SLA info for each endpoint.

Terms of Service

It is recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of the Using NDe APIs guide on how to create and register your caller ID.

Authorization

Access Tokens

Most calls through the Nike API gateway (api.nike.com) require an access token be sent in the request header. This allows Nike to verify that your app is authorized to perform the action on behalf of the user.

Access tokens are obtained by calling Nike Unite services prior to calling the API which you ultimately want to reach.

To find out more on how to call Unite services to obtain access tokens, see the Authorization section of the Using NDe APIs guide.

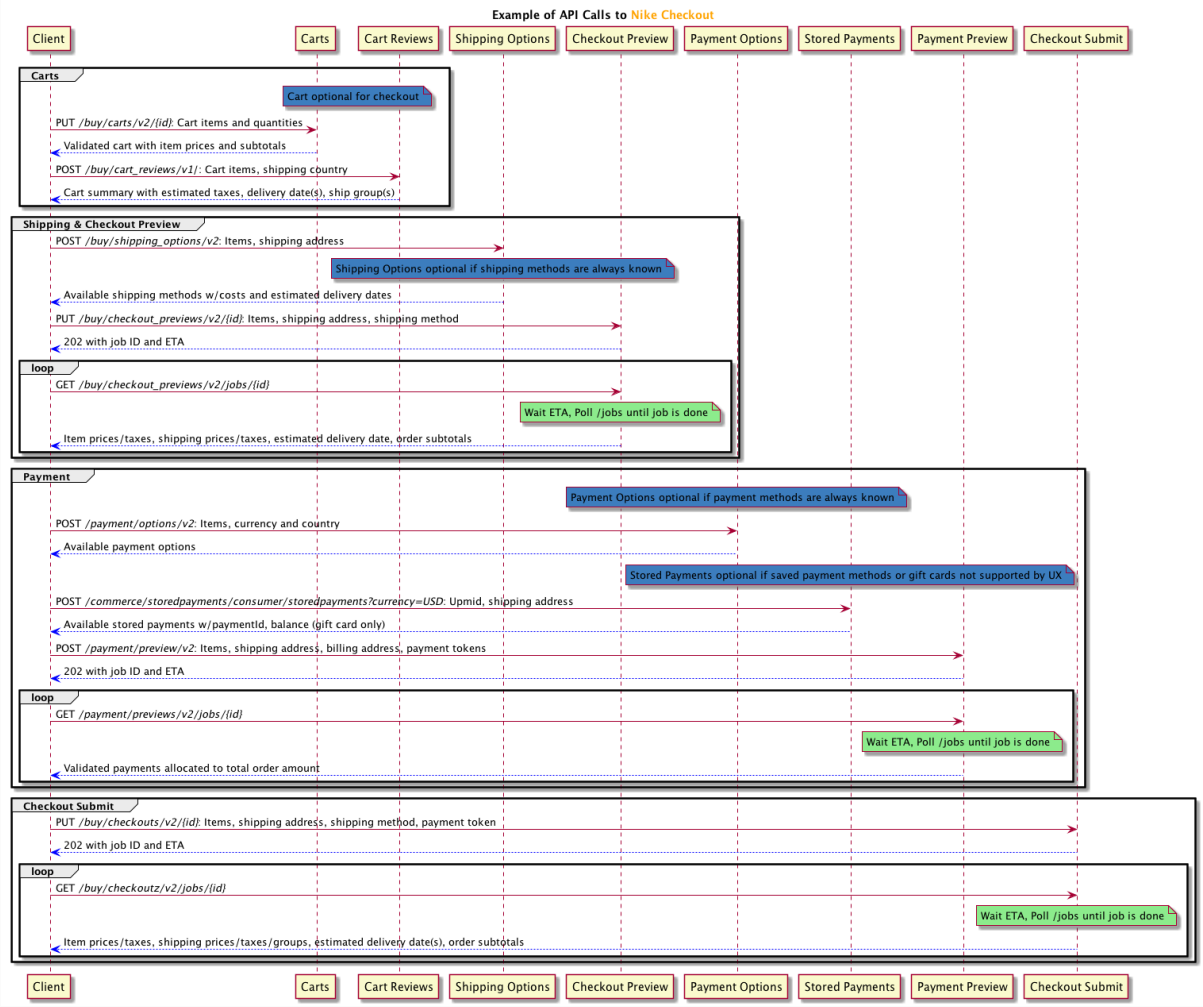
JSON Web Token

Only one endpoint in the Buy APIs, *Launch Checkout Submit*, requires the additional authorization of a JSON Web Token (JWT). For more, see the JWT section of Using NDe APIs.

Listed below are ways to troubleshoot unexpected responses using this API.

Example Implementation Diagram

Here is an example of a sequence of API calls to execute an entire checkout:



Sample Requests

The sample requests included throughout this guide often contain unique IDs and access tokens that are spent/expired in the Production environment, so you will not be able to copy and use them as-is for testing purposes. However, since other values in the request may still be valid and thus reusable, it may be convenient for you to copy the samples and change only the spent/expired values.

User Types

The Nike Checkout APIs support 3 distinct user types:

- Member: user has logged in with their Nike+ account credentials
- Guest: user has not logged in (anonymous user)
- Employee: user is an employee of Nike or a subsidiary and has logged in with swoosh.com credentials (also known as Swoosh user type)

Depending on user type, certain aspects of the calls that you make to the Checkout APIs might need to be modified. This will be called out whenever applicable in the detailed endpoint sections which follow in this guide. Also, consider that not all user types might apply to your app (e.g. you might only support Members).

See the User Types section of the Using NDe APIs guide for more information.

Idempotence

Idempotence means that the result of a successful request is independent of the number of times it is executed. What does that mean for the Checkouts API? Let's break it down.

Each PUT request to *Request Checkout Preview* and *Request a Checkout Submit* includes 1) a client-generated UUID (checkout ID) in the URL and 2) an Entity in the request body.

There are 4 possible scenarios:

Scenario	**Result**
UUID and Entity are new to the system (base use case)	Client receives HTTP 202 response, request processed as new job
UUID and Entity match a prior request	Client receives the exact same HTTP 202 response from the prior request (no new job processed)
UUID used previously, Entity is new	Client receives HTTP 409 error response (no new job processed)

Scenario	Result
UUID is new, Entity previously submitted under another UUID	Client receives HTTP 202 response, request processed as new job

TIP: For more, see the Idempotence Guarantee section of the Using NDe APIs guide.

Use Troubleshooting Tools

- Use the general troubleshooting tips in the Using NDe APIs guide.
- Use a Splunk query (requires access) to check for issues with your request.
- Contact the Buy team on the #cic-order-integration Slack channel for assistance.

Common Questions

Is it okay to call Checkout APIs if my app is hosted in an Amazon Web Services VPC?

Yes. The APIs are exposed publicly so it shouldn't matter where you are calling from. If you are calling repeatedly from a small set of IP addresses, it might be possible that Nike's bot-mitigation tools could interfere with your ability to make calls. If you are having issues, reach out to us for help.

My Checkout Submit job is taking a long time to complete. What gives?

Checkout Submits initiate a lot of behind-the-scenes API calls, the duration of which is somewhat unpredictable. Depending on the total volume of requests happening at the time your request was submitted, combined with the payment method and shipping country selected by the user, it may take several seconds to get a completed job. Best case is about 5 seconds, worst case can be well over a minute. If the job times out, you will get a 'completed with error' job status.

Glossary

See the Glossary

Document Change Log

Summary	Date	Description
Initial draft	08/30/2018	Initial Draft

Related Links

NDe Documentation Home

Getting Started

Business Guides

Developer's Guides